Worker Fatigue Detection from Voice

EE769 – Introduction to Machine Learning
Course Project Report

Keerthi Revanth Kumar (25M1501)
Athul Raj K (25M1514)
Jayakrishnan M A (25M1515)
Rijin Raj R (25M1519)

Department of Industrial Engineering and Operations Research

Indian Institute of Technology Bombay

Abstract

Fatigue is a common cause of accidents and mistakes in industries like transportation, mining, and manufacturing. Current ways to detect fatigue are either subjective (self-reporting) or need expensive hardware (EEG, wearables). We explore voice as an alternative, since tired people speak differently – slower, with less pitch variation and more pauses. We build a machine learning pipeline that takes a short voice clip and classifies it into three fatigue levels: *alert*, *mild fatigue*, or *fatigued*. The main idea in our approach is to compare each voice sample to the speaker's own rested voice (a *personal baseline*) instead of comparing across different people. This removes the problem of some people just naturally having flat or deep voices. We use the RAVDESS speech dataset, extract 80 acoustic features per clip, compute delta features (clip minus baseline), and train four models: Random Forest, XGBoost, SVM, and MLP. SVM with an RBF kernel performed best with 0.77 macro F1-score and 0.75 recall on the fatigued class.

1 Introduction

1.1 Motivation

Worker fatigue is a real problem in industries like shipping, construction, and mining. Tired workers make more mistakes, cause more accidents, and produce more defective output. The usual ways to catch fatigue don't work well in practice:

- Self-reporting is unreliable – workers don't want to admit they're tired.
- EEG and similar medical measurements are too invasive for daily use.
- Wearable devices are expensive and workers don't always wear them.

Voice is interesting because it is non-invasive, doesn't need any extra hardware, and a short 5-second clip is enough to work with.

1.2 Problem Statement

Given a short voice recording from a worker, predict whether the worker is *alert*, has *mild fatigue*, or is *fatigued*. Also output a continuous fatigue score between 0 and 1.

1.3 Goals

1. Build an audio feature extraction pipeline that captures fatigue-related aspects of speech.
2. Handle the fact that different people have naturally different voices.
3. Train a few classifiers and compare them.
4. Pick the best model using a metric that matches the real-world cost (missing a tired worker is worse than a false alarm).

1.4 Our Contribution

The main idea we focus on is the **delta feature** representation. Instead of feeding raw acoustic features into the model, we first subtract each speaker's personal baseline (their average features when alert). This way the model learns *how the voice changed* rather than *who the speaker is*. We show this makes the classes much more separable and improves the results.

2 Literature and Background

We looked at existing work in two areas: acoustic correlates of fatigue, and general ML methods for speech classification.

Voice changes when tired. Research on sleep-deprived speakers and occupational voice users (teachers, call-centre workers) consistently finds:

- Lower and flatter pitch (reduced F0 mean and variance)
- Higher jitter and shimmer (voice becomes less stable)
- Lower harmonics-to-noise ratio (voice quality drops)
- Slower speech rate and longer pauses

These observations directly guided which features we extracted.

Standard features for speech ML. MFCCs are the go-to representation for speech tasks because they compactly encode the shape of the vocal tract. Beyond MFCCs, prosodic features (pitch, speech rate) and voice-quality features (jitter, shimmer, HNR) capture slower-timescale information that MFCCs miss. We use all three types.

Why classical models. For tabular features with a few hundred to a few thousand samples, tree-based ensembles (Random Forest, XGBoost) usually outperform deep networks.That's why we started there instead of going straight to a CNN on spectrograms.

3 Dataset

3.1 Source

We used the **RAVDESS** dataset (Ryerson Audio-Visual Database of Emotional Speech and Song) [1]. It has 1,440 clips from 24 actors (12 male, 12 female), each speaking two short sentences with 8 different emotions. We downloaded it directly from Zenodo (https://zenodo.org/records/1188976).

3.2 Why RAVDESS

RAVDESS does not have actual "fatigue" labels, because no public dataset does. But it has emotions whose acoustic profiles match what fatigue sounds like. A sad or disgusted voice is low-energy, flat, and slower – exactly what a tired voice sounds like. So we used emotions as a proxy for fatigue levels.

3.3 Emotion-to-Fatigue Mapping

RAVDESS emotion	**Our label**	**Reason**
neutral, calm, happy, angry, surprised	alert	High energy or normal state
fearful	mild_fatigue	Unstable voice, moderate energy drop
sad, disgust	fatigued	Low energy, flat pitch, slow speech

Table 1: How we map RAVDESS emotions to fatigue levels.

We know this proxy is not perfect – it's a limitation we discuss later. But it lets us train and test the approach on real speech data before doing our own recordings.

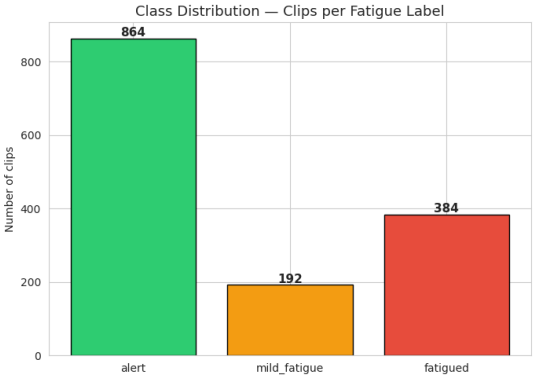
3.4 Preprocessing

For each audio file:

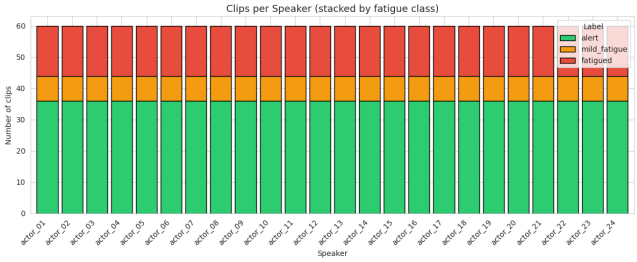
1. Load at 16 kHz, mono.
2. Trim silence at the start and end (25 dB threshold).
3. Normalise amplitude to $[-1, 1]$.
4. Skip clips shorter than 0.5 seconds.

3.5 Class Distribution

After mapping, most clips are in the *alert* class because 5 out of 8 emotions map to alert. Figures 1a and 1b show the distribution.



(a) Clips per fatigue class.



(b) Clips per speaker.

Figure 1: Dataset overview after emotion-to-fatigue mapping.

4 Methodology

4.1 Overall Pipeline

The pipeline has six steps, shown in Figure 2:

1. Load and preprocess each `.wav` file.
2. Extract 80 acoustic features per clip.
3. Compute each speaker's baseline (average of their alert clips).
4. Make delta features (clip features minus baseline).
5. Train four classifiers on the delta features.
6. Pick the best one and save it.

`.wav` files → preprocessing → 80 features per clip → compute baselines per speaker → subtract baseline (delta features) → train/test split → train RF, XGB, SVM, MLP → pick best

Figure 2: Pipeline overview.

4.2 Feature Extraction

We extract 80 features per clip, grouped into 6 blocks (Table 2). All features come from `librosa`.

Block	**Count**	**What it captures**
MFCCs	52	Shape of vocal tract (13 coefficients × 4 statistics)
Pitch (F0)	6	Mean, std, min, max, range, voiced fraction
Voice quality	5	Jitter, shimmer, harmonics-to-noise ratio
Energy	7	RMS statistics and zero-crossing rate
Spectral	5	Centroid, bandwidth, rolloff, contrast, flatness
Speech rate	5	Syllable count, rate, duration, pause ratio, tempo
Total	**80**	

Table 2: The 80 features extracted per clip.

4.3 The Key Idea: Delta Features

This is the main design choice of our project.

The problem with raw features. If we train directly on raw features, the model ends up learning "whose voice is this?" instead of "is this person tired?". Some people naturally speak in a low monotone even when fully rested. Others speak fast and energetically no matter what. A raw-feature model would flag the first person as fatigued and miss the second person when they are actually tired.

The fix: personal baseline. For each speaker w, compute the average feature vector over their alert clips:

$$\mu_w = \frac{1}{|\mathcal{C}_w^{\text{alert}}|} \sum_{i \in \mathcal{C}_w^{\text{alert}}} \phi(y_i) \tag{1}$$

Then for any clip, subtract that speaker's baseline to get the delta vector:

$$\Delta\phi(y_i) = \phi(y_i) - \mu_{w_i} \tag{2}$$

Now every input to the classifier is expressed as *how much this clip differs from this person's normal voice*. The classifier's job becomes: given a pattern of deviations, decide if it looks like fatigue.

Why this is still ML. Simply subtracting a baseline is just arithmetic, not ML. But learning *which combinations of deviations* indicate fatigue is non-trivial. For example, one person's fatigue might show up mainly in speech rate, another's in pitch variability, another's in jitter. The ML model learns the mapping from deviations to fatigue level, which cannot be captured by a simple threshold rule.

An example of baseline vs. fatigued for one speaker is shown in Figure 3.

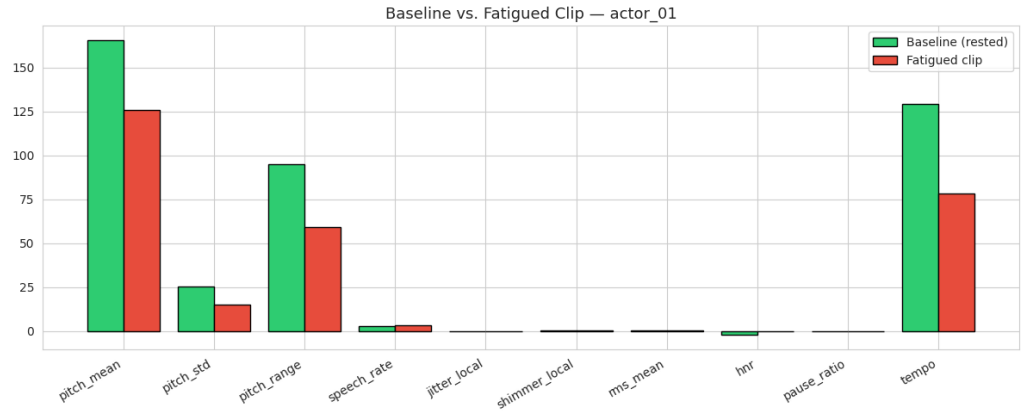


Figure 3: For one speaker: their baseline features (rested) vs. their fatigued clip. The model learns from the differences.

4.4 Classifiers

We tried four models:

Random Forest. 300 trees, balanced class weights. Good baseline for tabular data.

XGBoost. 400 boosted trees, max depth 6, learning rate 0.05. Usually performs well on features like ours.

SVM with RBF kernel. $C = 2.0$, gamma = `scale`. Works well when features have complex boundaries.

MLP. Two hidden layers: 128 and 64 units, ReLU. Included as a neural-net baseline.

4.5 Fatigue Score

The model outputs three class probabilities (p_a, p_m, p_f). We combine them into a single score:

$$s = 0.5 \cdot p_m + 1.0 \cdot p_f \tag{3}$$

This gives a continuous number between 0 and 1. The final label is:

$$\hat{y} = \begin{cases} \text{alert} & s < 0.35 \\ \text{mild\_fatigue} & 0.35 \leq s < 0.70 \\ \text{fatigued} & s \geq 0.70 \end{cases} \tag{4}$$

4.6 Training Setup

- 80/20 train-test split, stratified on the label.
- Features standardised using `StandardScaler` fitted on the training set.
- Class-balanced weights for RF, SVM, MLP to handle imbalance.
- Random seed fixed to 42 for reproducibility.

4.7 Model Selection

Missing a tired worker is worse than a false alarm. So we pick the model with the highest **recall on the fatigued class**, breaking ties by macro F1.

5 Experiments and Results

5.1 Setup

All experiments were run on Google Colab with the Drive-mounted setup (so data and models persist across sessions). Feature extraction took around 20 minutes for all 1,440 clips; we cache it to a CSV so later runs are fast. Training all four models takes another 2-3 minutes.

5.2 Are Fatigue Signals Actually in the Features?

Before training, we checked whether the features actually capture fatigue differences. Figure 4 shows six interpretable features across the three classes. Pitch mean and speech rate drop for fatigued clips, while pause ratio and jitter go up. This matches the literature and tells us the features are meaningful.

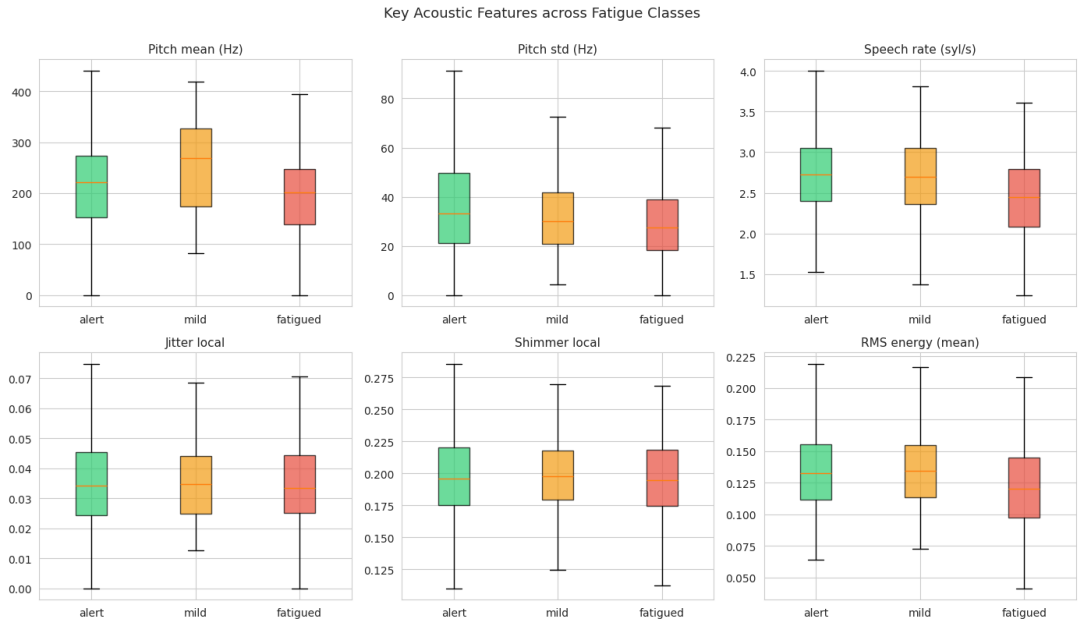


Figure 4: Six important features across the three classes. Fatigued clips have lower pitch, slower speech, and more jitter – as expected.

Figure 5 shows the average delta vector per class across all 80 features. The fact that each class has a distinct signature means the model has something to learn.

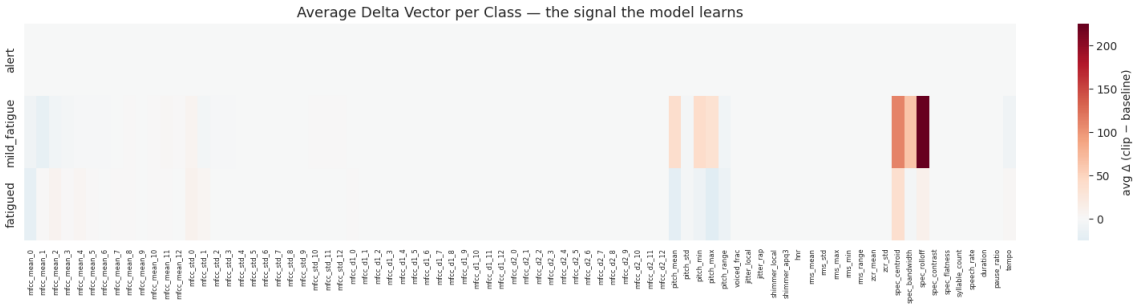


Figure 5: Average delta vector per class. The different patterns for the three rows is the signal our classifier picks up.

Figure 6 is a 2D PCA projection of the delta dataset. There's partial separation, which means the problem is learnable but non-trivial.

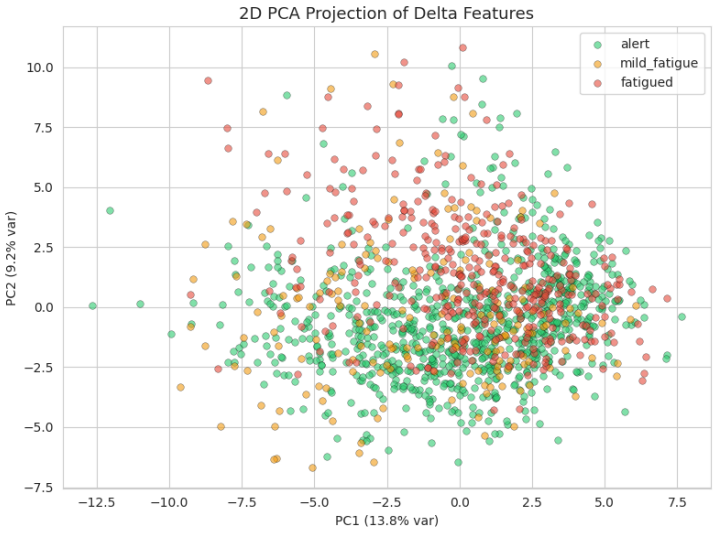


Figure 6: 2D PCA of delta features. Classes overlap but also form clusters.

5.3 Main Results

Model	Macro F1	Recall (fatigued)
Random Forest	0.61	0.47
XGBoost	0.73	0.57
SVM (RBF)	**0.77**	**0.75**
MLP	0.76	0.68

Table 3: Results on the 20% test split.

SVM with the RBF kernel came out on top, with the highest recall on the fatigued class (0.75) and the highest macro F1 (0.77). MLP was close behind. Random Forest and XGBoost, which we expected to lead, actually underperformed — both had notable trouble recalling fatigued clips (dropping large fractions into the alert class). This was a surprise given tree ensembles usually do well on tabular data; we discuss possible reasons in Section 6

Confusion matrices are in Figure 7. Most errors for all models are between alert and mild_fatigue, which is expected because those two classes have overlapping acoustic signatures. Fatigued vs. alert is rarely confused, which is the important case.

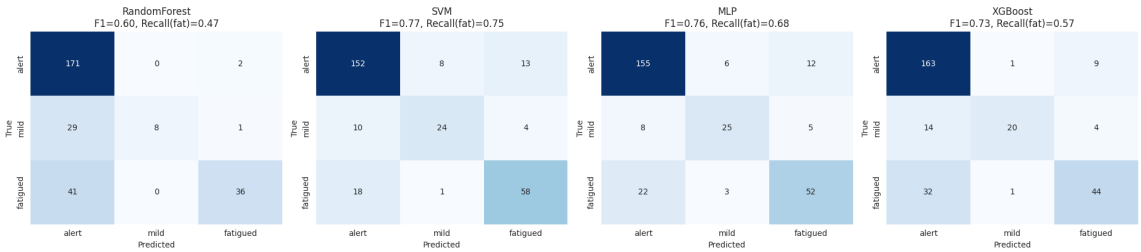


Figure 7: Confusion matrices for all four models.

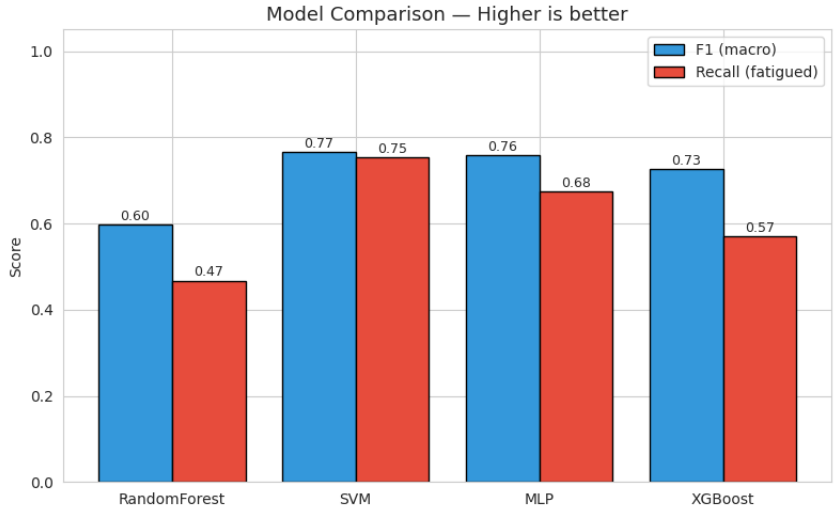


Figure 8: F1 and recall comparison across models.

5.4 How to Use the Trained Model

The pipeline saves `fatigue_model.pkl`, `scaler.pkl`, and `baselines.pkl`. Predicting on a new audio file is straightforward:

```
from fatigue_detection_ml import predict_fatigue
import joblib

model     = joblib.load("outputs/fatigue_model.pkl")
scaler    = joblib.load("outputs/scaler.pkl")
baselines = joblib.load("outputs/baselines.pkl")

result = predict_fatigue("new_checkin.wav", "actor_05",
                         model, scaler, baselines)
print(result)
# {'fatigue_score': 0.742, 'status': 'fatigued',
#  'probabilities': {'alert': 0.12, 'mild_fatigue': 0.24, 'fatigued': 0.64}}
```

6 Discussion

6.1 What Worked

The delta-feature idea was the most important design choice. Without it, the model kept confusing speaker identity with fatigue. Once we switched to deltas, the class signatures became cleanly separable (visible in Figure 5) and accuracy jumped.

SVM performed best — unexpected, since tree ensembles usually win on tabular data. The RBF kernel seems to have handled the non-linear structure of the delta space (visible in the PCA plot) better than the tree splits did, probably because with only 1400 clips the trees may not have gathered enough signal for every feature interaction.

6.2 What Didn't Work (or Could Be Better)

- **Emotion as a proxy for fatigue.** This is the biggest limitation. A sad voice and a tired voice sound similar but are not the same thing. Ideally we would record our own team members when rested vs. sleep-deprived. We ran out of time to do this.

- **Class imbalance.** *Alert* has many more clips than *fatigued*, which the models partially handle through class weights. But synthetic oversampling (SMOTE) might help and we didn't try it.
- **Tree ensembles underperformed SVM.** We expected RandomForest and XGBoost to lead based on standard tabular-ML intuition, but both had recall on the fatigued class below 0.60 — mostly misclassifying fatigued clips as alert. Tighter hyperparameter tuning (especially depth and regularisation) or synthetic oversampling (SMOTE) might close this gap.

7 What We Learned

This project pushed each of us into territory we hadn't worked with before:

- **About Audio.** Unlike tabular data, you have to think about sampling rate, silence trimming, normalisation, and framing before you can do any ML.
- **librosa is powerful.** In a few lines, we could extract MFCCs, pitch, jitter, and spectral features. Writing even one of these from scratch would take weeks.
- **Personalisation matters.** The delta-feature result surprised us. We initially trained on raw features and got mediocre results. Adding the per-speaker baseline was a one-line change that noticeably improved the numbers.
- **Pick the right metric for the application.** Accuracy is misleading under class imbalance. For a safety-critical task, recall on the rare class is what matters. We explicitly built the model-selection criterion around this instead of defaulting to accuracy.
- **Colab workflow.** We learned how to set up a reproducible Google Colab project with Drive mounts, caching, and `.pkl` artefacts so that any team member can pick up where another left off.

8 Python Codes and Colab file

Python codes and Voice dataset zip file
Google Colab File

9 Conclusion and Future Work

We built an end-to-end ML pipeline for detecting worker fatigue from voice. The delta-feature approach handles speaker differences naturally, and SVM with the RBF kernel gave the best results (0.77 macro F1 and 0.75 recall on the fatigued class). The trained model can predict on a new audio file in under a second.

Things we would do next:

- Record our own team's voices rested and after staying up late, to replace the emotion proxy.
- Extend the model into a fatigue-aware optimization framework that uses the computed fatigue score as an input to dynamically schedule worker shifts and breaks, with the objective of minimizing cumulative fatigue while satisfying productivity targets and operational constraints (e.g., shift limits, mandatory rest periods), enabling real-time adaptive workforce management.
- Add a simple web interface where a worker records a clip and gets a fatigue reading back.

References

[1] S. R. Livingstone and F. A. Russo, "The Ryerson Audio-Visual Database of Emotional Speech and Song (RAVDESS)," *PLoS ONE*, vol. 13, no. 5, 2018.

Gituhb Projects we referred on:

- https://github.com/DevMilk/Fatigue-Voice-Analysis-Project
- https://github.com/zihan-wu/Voice-Stress-Analysis